



Deep Learning Assignment 1

Syeda Eman Ali

DS 7A

22i-1936

Submitted to : Dr. Qurat ul Ain

Date 27/9/2025

1. Pre-processing & Data Examination

DL_Assignment1_Dataset

DL_Assignment1_Dataset1

DL_Assignment1_Dataset2

DL_Assignment1_Dataset3

Dataset

sample_data

DL_Assignment1_Dataset.zip

model.png

DL_Assignment1_Dataset

DL_Assignment1_Dataset1

DL_Assignment1_Dataset2

DL_Assignment1_Dataset3

Dataset

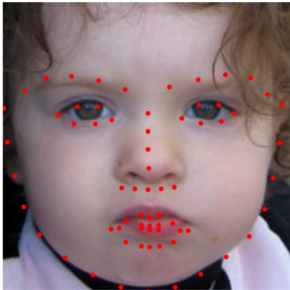
sample_data

DL_Assignment1_Dataset.zip

model.png

```
plt.imshow(img)
plt.scatter(lnd[:,0], lnd[:,1], c="r", s=10)
plt.axis("off")
plt.show()
```

Expression: 0
Valence: -0.176846
Arousal: -0.0776398
Landmarks Before Re-shape: (136,)
Landmarks shape after reshape: (68, 2)



idx = 565

DL_A1.ipynb

Runtime Tools Help

Run all

RAM
Disk

set

set1

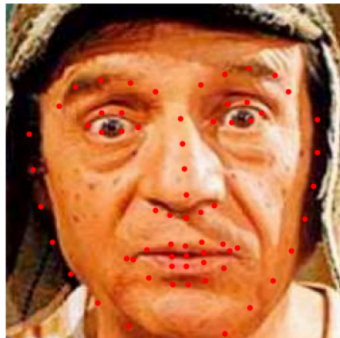
set2

set3

set.zip

```
plt.imshow(img)
plt.scatter(lnd[:,0], lnd[:,1], c="r", s=10)
plt.axis("off")
plt.show()
```

Expression: 4
Valence: -0.142857
Arousal: 0.84127
Landmarks Before Re-shape: (136,)
Landmarks shape after reshape: (68, 2)



I. Data Cleaning / Filtering

- Collected all image IDs and corresponding expression labels.
- Removed samples where any label had value -2 (uncertain or no-face).
- Filtered `ids_to_keep` for a clean dataset.

Found 3999 valid IDs

Sample IDs: ['739', '4705', '4865', '982', '4509', '5191', '2633', '4131', '5388', '2946']

```
ids_to_not_keep = []

for img_id in img_ids:
    exp = np.load(os.path.join(ann_dir, f"{img_id}_exp.npy"))
    val = np.load(os.path.join(ann_dir, f"{img_id}_val.npy"))
    aro = np.load(os.path.join(ann_dir, f"{img_id}_aro.npy"))

    if -2 in [exp, val, aro]:
        ids_to_not_keep.append(img_id)

print("Number of samples with -2:", len(ids_to_not_keep))
print("Examples of IDs with -2:", ids_to_not_keep[:20])
```

Number of samples with -2: 0
Examples of IDs with -2: []

II. Dataset Splitting

1. Split the dataset into **train (70%)** and **temp (30%)**.
2. Split temp into **validation (15%)** and **test (15%)**.

Stratified split ensures equal class distribution across splits.

```
print("Train size:", len(train_ids))
print("Val size:", len(val_ids))
```

Train size: 2799
Val size: 600

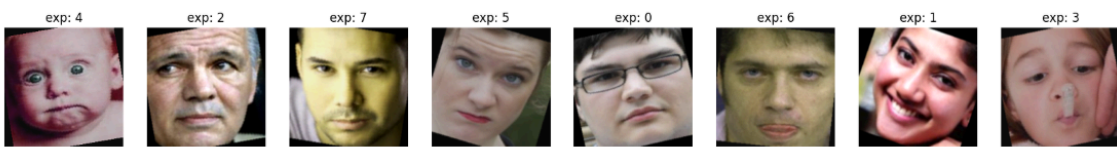
III. Data Augmentation

```
[18]: import matplotlib.pyplot as plt

images, exps, vals, aros = next(iter(train_loader))

fig, axes = plt.subplots(1, 8, figsize=(20, 5))
for i in range(8):
    img = images[i].permute(1, 2, 0).numpy()
    axes[i].imshow(img)
    axes[i].set_title(f"exp: {exps[i].item()}")
    axes[i].axis("off")

plt.show()
```



Data Augmentation

[14]
✓ Os

```
# transforms
train_aug = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(20),
    transforms.RandomResizedCrop(224, scale=(0.8, 1.0)),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1),
    transforms.RandomAdjustSharpness(sharpness_factor=2, p=0.3),
    transforms.GaussianBlur(kernel_size=3, sigma=(0.1, 2.0)),
    transforms.ToTensor(),
    transforms.RandomErasing(p=0.2)
])

valid_aug = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor()
])
```

Training transforms:

- Resize, random flip, rotation, crop, color jitter, sharpness, Gaussian blur, random erasing.

Validation transforms:

- Resize and convert to tensor only (no augmentation).

Model Architecture

Model: "efficientnetb0"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 224, 224, 3)	0	-
rescaling (Rescaling)	(None, 224, 224, 3)	0	input_layer[0][0]
normalization (Normalization)	(None, 224, 224, 3)	7	rescaling[0][0]
rescaling_1 (Rescaling)	(None, 224, 224, 3)	0	normalization[0]...
stem_conv_pad (ZeroPadding2D)	(None, 225, 225, 3)	0	rescaling_1[0][0]
stem_conv (Conv2D)	(None, 112, 112, 32)	864	stem_conv_pad[0]...
stem_bn (BatchNormalizatio...	(None, 112, 112, 32)	128	stem_conv[0][0]
stem_activation (Activation)	(None, 112, 112, 32)	0	stem_bn[0][0]
block1a_dwconv	(None, 112, 112, 32)	288	stem_activation[...

- Built mainly using **Depthwise Separable Convolution blocks (MBConv)** → makes it lightweight and efficient.
- Uses **Squeeze-and-Excitation (SE) modules**, which **analyze each channel and boost the important ones while suppressing the weak ones**, helping the network *focus on more useful features*.
- Uses **Swish / SiLU activation** instead of ReLU for smoother gradients and better accuracy.
- Mostly relies on **3×3 and 5×5 convolutions**.
- **Batch Normalization** is applied after most layers for stability.
- Ends with **Global Average Pooling** before the final classification layer.
- Starts with a **single stem convolution**, followed by **stacked MBConv blocks** with more filters as depth increases.

- Designed with **balanced scaling**, meaning width, depth, and resolution grow together efficiently.

Dataset Class (EmotionDataset)

- Custom PyTorch Dataset class:
 - Loads image and applies transforms.
 - Loads expression, valence, arousal labels as tensors.
- Created **DataLoaders** for train, validation, and test sets with batch size 32.

Model Setup

- Used **pretrained CNNs** (EfficientNetB0 & Mobilenetv3).
- Modified classifier heads for multi-task learning:
 - Expression classification (8 classes)
 - Valence regression (1 value)
 - Arousal regression (1 value)
- Send the model to the GPU if available.

Loss & Optimizer

- Classification: CrossEntropyLoss
- Regression: MSELoss (for valence & arousal)
- Optimizer: Adam with learning rate 1e-4

Training Loop (Trainer Class)

- Tracks history: train/val loss & accuracy, regression metrics, classification metrics.
- Performs **multi-task training**:
 - Forward pass
 - Compute combined loss: classification + valence + arousal
 - Backward pass & optimizer step
- Validation at each epoch with metrics calculation:
 - **Regression**: RMSE, correlation, CCC, SAGR
 - **Classification**: F1, Cohen's Kappa, Krippendorff's Alpha, ROC-AUC, PR-AUC
- Implements **early stopping** based on validation loss.

1. Efficient Net b0

[18]
✓ 3s

```

 IMG_SHAPE = (224, 224, 3)
model0 = tf.keras.applications.EfficientNetB0(input_shape=IMG_SHAPE, include_top=False, weights="imagenet")
tf.keras.utils.plot_model(model0)
model0.summary()

```

 Model: "efficientnetb0"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 224, 224, 3)	0	-
rescaling (Rescaling)	(None, 224, 224, 3)	0	input_layer[0][0]
normalization (Normalization)	(None, 224, 224, 3)	7	rescaling[0][0]
rescaling_1 (Rescaling)	(None, 224, 224, 3)	0	normalization[0]...
stem_conv_pad (ZeroPadding2D)	(None, 225, 225, 3)	0	rescaling_1[0][0]
stem_conv (Conv2D)	(None, 112, 112, 32)	864	stem_conv_pad[0]...
stem_bn (BatchNormalizatio...	(None, 112, 112, 32)	128	stem_conv[0][0]
stem_activation (Activation)	(None, 112, 112, 32)	0	stem_bn[0][0]
block1a_dwconv (DepthwiseConv2D)	(None, 112, 112, 32)	288	stem_activation[...
block1a_bn (BatchNormalizatio...	(None, 112, 112, 32)	128	block1a_dwconv[0...
block1a_activation	(None, 112, 112, 32)	0	block1a_bn[0][0]

```

)
(avgpool): AdaptiveAvgPool2d(output_size=1)
(classifier): MultiHead(
  (expression): Linear(in_features=1280, out_features=8, bias=True)
  (valence): Linear(in_features=1280, out_features=1, bias=True)
  (arousal): Linear(in_features=1280, out_features=1, bias=True)
)
)

```

Altered the final classification layer to include 3 heads, 2 for regression (valence and arousal) and the 3rd one for classifying emotion.

Training EfficientNet with Transfer Learning

- A pretrained EfficientNet model is used as the backbone so the network starts with strong image understanding rather than learning from scratch.
- Each image is passed through EfficientNet's feature extractor, and the output is pooled into a fixed-length feature vector.
- On top of that, **custom heads are added**, one for classification (expression) and two for regression (valence and arousal).
- Training runs for up to 60 epochs, but early stopping with a patience of 10 is used so training ends automatically if validation loss stops improving.
- A ReduceLROnPlateau scheduler lowers the learning rate when progress slows down, helping with finer convergence in later epochs.
- The model is optimized using a combination of Cross-Entropy (for expression) and MSE losses (for valence and arousal), with adjustable weighting for both regression targets.
- Key metrics like Accuracy, F1, Kappa, RMSE, CCC, ROC-AUC and more are calculated every epoch for a full performance overview.

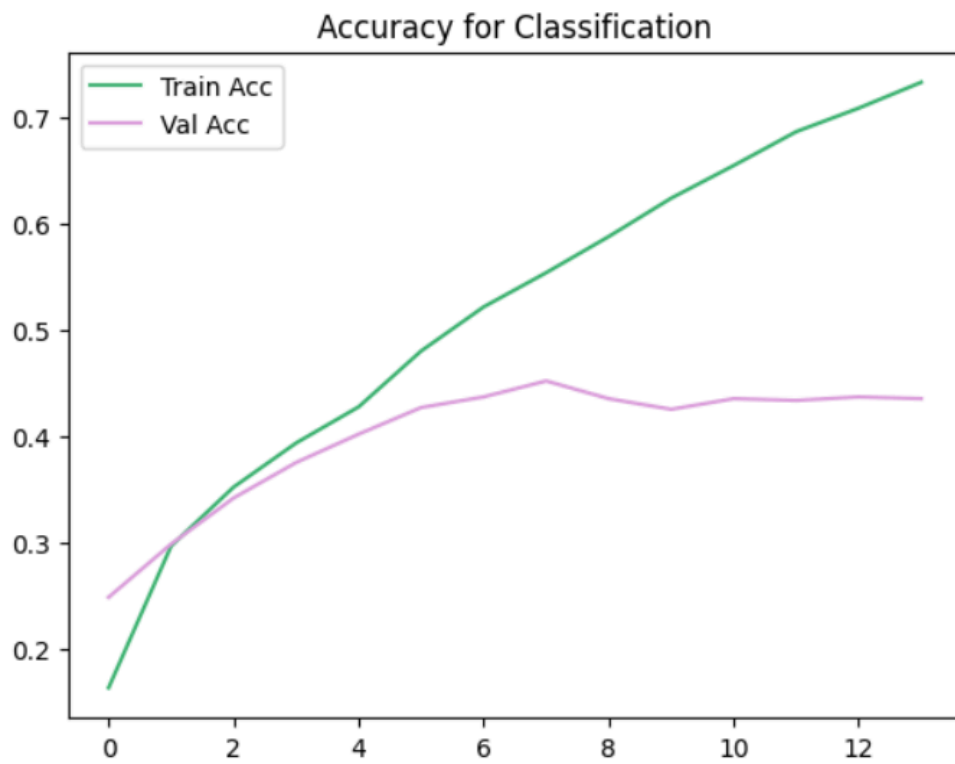
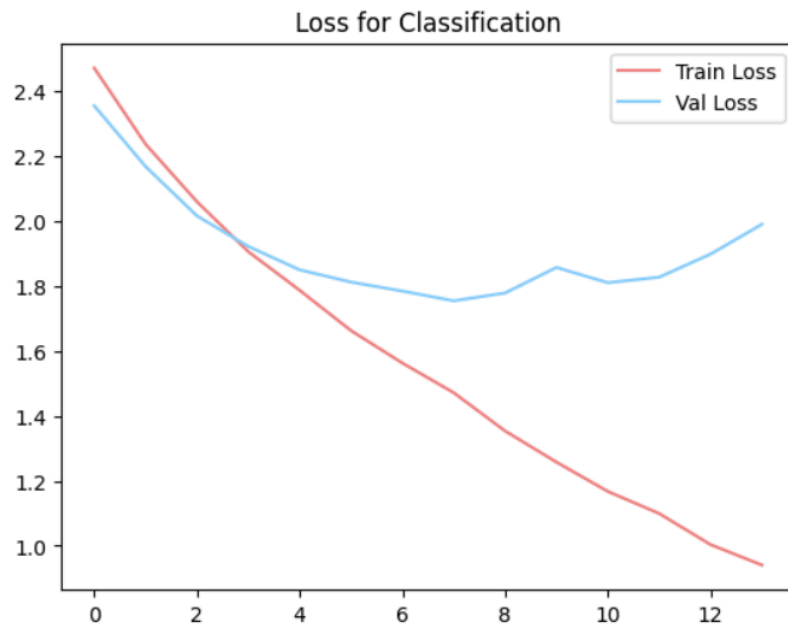
$(+ \text{ Code}) \text{ --- } (+ \text{ Text})$

 Share
 Gemini

zip

```
Train set size: 2799
Valid set size: 600
Test set size: 600
Batch size: torch.Size([32, 3, 224, 224])
Batches per epoch: 88
```

```
plt.show()
```



1	2.036424	1.914815	0.299750	0.325000	0.296329	0.228571	0.115527	0.766644	0.339489
2	1.851534	1.748476	0.365488	0.400000	0.390519	0.314286	0.271065	0.807429	0.416479
3	1.689279	1.653642	0.436584	0.450000	0.442661	0.371429	0.297007	0.824911	0.456127
4	1.569617	1.601729	0.468382	0.471667	0.461786	0.396190	0.349834	0.838759	0.479581
5	1.446106	1.600033	0.514469	0.450000	0.443424	0.371429	0.341758	0.838705	0.473815
6	1.355681	1.582310	0.545909	0.466667	0.461559	0.390476	0.336620	0.843286	0.484994
7	1.257364	1.553583	0.580564	0.485000	0.479310	0.411429	0.330510	0.851403	0.496054
8	1.143786	1.596477	0.630225	0.475000	0.472114	0.400000	0.347625	0.848387	0.495062
9	1.096837	1.651094	0.640943	0.470000	0.464534	0.394286	0.345583	0.843149	0.482774
10	1.036182	1.643395	0.663809	0.470000	0.467189	0.394286	0.316833	0.844019	0.491189
11	0.936378	1.721790	0.712397	0.468333	0.466990	0.392381	0.337808	0.840146	0.481865
12	0.838930	1.715468	0.731690	0.478333	0.472238	0.403810	0.359346	0.842679	0.488376
13	0.773302	1.739576	0.762415	0.478333	0.471457	0.403810	0.348278	0.840571	0.487800
14	0.750653	1.769987	0.767774	0.481667	0.481238	0.407619	0.329688	0.840283	0.486863
15	0.734702	1.775959	0.770632	0.478333	0.477515	0.403810	0.345837	0.838997	0.491089
16	0.674188	1.830407	0.799214	0.470000	0.466308	0.394286	0.324703	0.836000	0.480390
17	0.642382	1.845220	0.803501	0.470000	0.463742	0.394286	0.326210	0.835159	0.475758

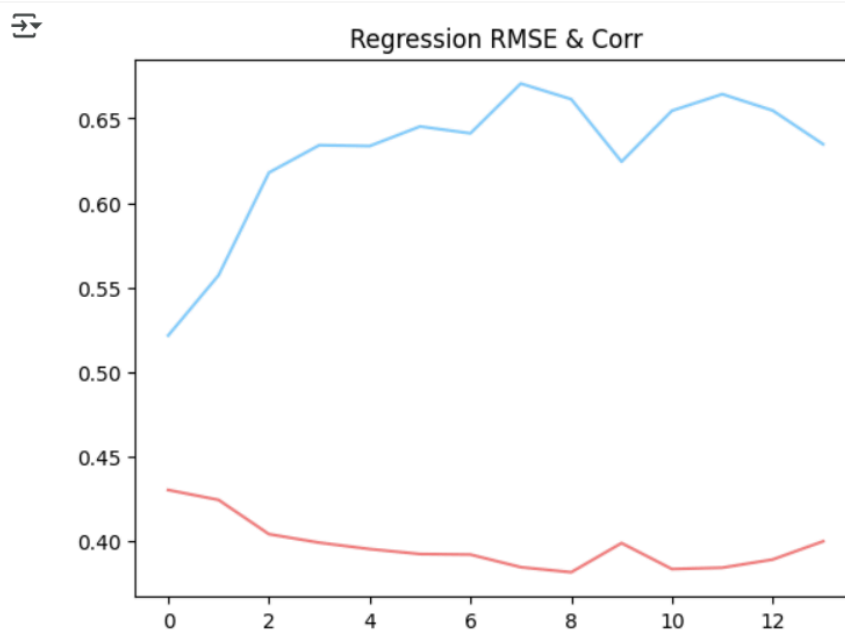
Classification Metrics

- Training accuracy increased from **0.18 → 0.80**.
- Validation accuracy increased from **0.26 → 0.48**.
- F1-score stabilized near **0.47**.
- Cohen's Kappa remained around **0.40** in later epochs.
- Krippendorff's Alpha remained **below 0.35**.
- ROC-AUC stayed above **0.83** after early epochs.
- PR-AUC stayed close to **0.48–0.49** in later epochs.

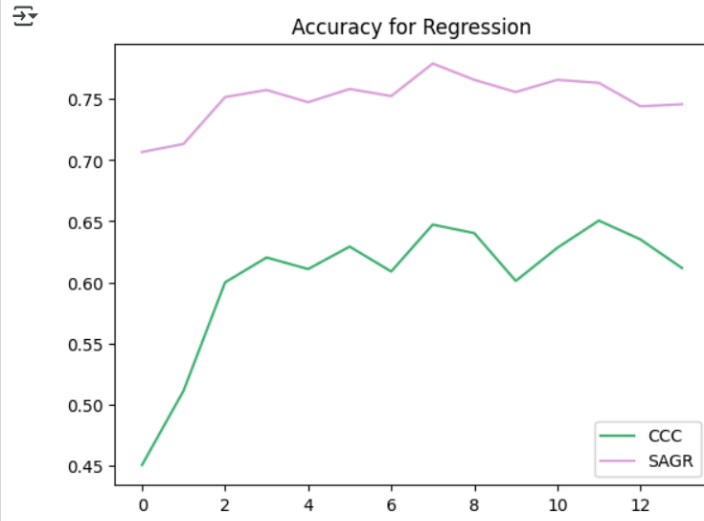
Regression Metrics

- RMSE reduced from **0.42** → **0.36**.
- Pearson correlation increased from **0.56** → **0.70**.
- CCC stabilized around **0.68–0.69**.
- SAGR remained around **0.75–0.77**.

```
plt.title("Regression RMSE & Corr")  
plt.show()
```

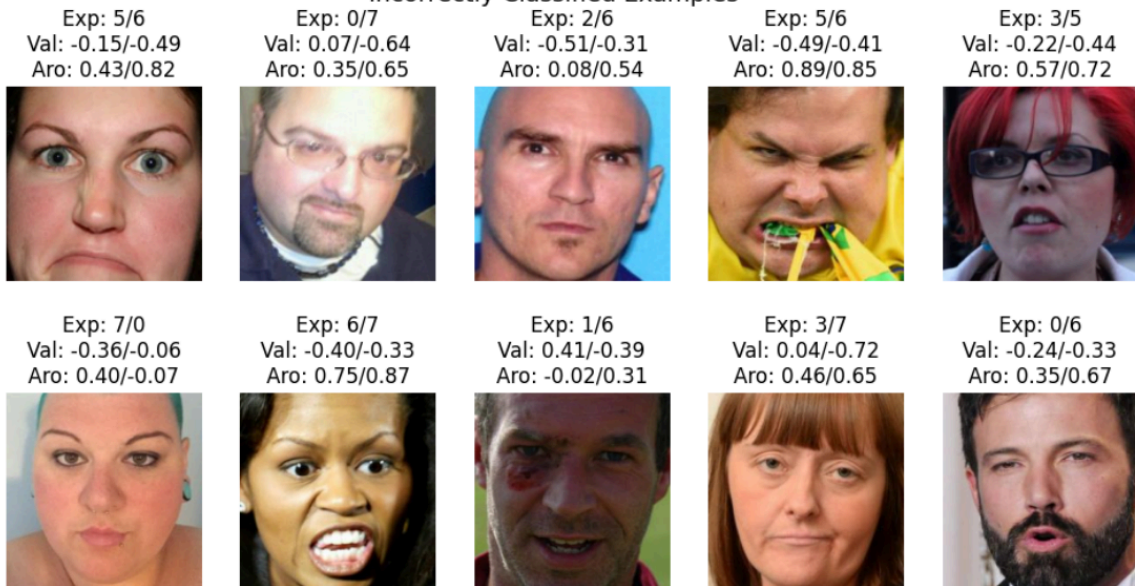



```
plt.title("Accuracy for Regression")  
plt.show()
```



	rmse	corr	ccc	sagr
0	0.420159	0.564039	0.486821	0.726667
1	0.400650	0.615688	0.569969	0.745000
2	0.383088	0.656220	0.599056	0.745000
3	0.379963	0.676560	0.660226	0.764167
4	0.371429	0.689275	0.673847	0.743333
5	0.374475	0.678192	0.652728	0.750833
6	0.367485	0.698034	0.680102	0.760833
7	0.357883	0.708351	0.679582	0.763333
8	0.367611	0.693694	0.668115	0.762500
9	0.367641	0.700364	0.670405	0.751667
10	0.363993	0.701952	0.681145	0.759167
11	0.372220	0.697512	0.688099	0.770000
12	0.366247	0.702646	0.691758	0.768333
13	0.363964	0.703864	0.689136	0.774167
14	0.368466	0.695982	0.681182	0.765833
15	0.364028	0.705185	0.690831	0.760833
16	0.369097	0.694360	0.678797	0.765833

Incorrectly Classified Examples



Correctly Classified Examples



- **Classification accuracy is reasonably good** which shows that the model is able to correctly classify nearly half of the samples across multiple categories.

- **F1-score and Cohen's Kappa (~0.47 and ~0.40)**, indicating that while the model is making correct predictions, it still has room to improve in class consistency and agreement.
- **ROC-AUC stays high (above 0.83)**, which means the model is good at ranking the correct class higher than the others even when final predictions are not always accurate.
- **PR-AUC near 0.48–0.49** shows a balanced trade-off between precision and recall but suggests that positive predictions are not always reliable.
- **Krippendorff's Alpha remains low (below 0.35)**, showing that agreement across all classes is inconsistent, some classes are likely performing better than others.

2. ResNet50

Model Architecture

- Uses standard 3×3 convolutional layers stacked sequentially.
- Introduces Residual Connections (Skip Connections) that allow the input of a block to be added directly to its output.
- These shortcuts help gradients flow easily during backpropagation, reducing vanishing gradient problems in deep models.
- Composed of Residual Blocks, each block has two or three conv layers + a skip connection.
- Uses Batch Normalization and ReLU after each convolution.
- Downsampling is performed using stride-2 convolutions or 1×1 projections in shortcuts.
- Ends with a Global Average Pooling layer followed by a Fully Connected Layer for classification..

```

ResNet50MultiTask(
  (backbone): ResNet(
    (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
    (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU(inplace=True)
    (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
    (layer1): Sequential(
      (0): Bottleneck(
        (conv1): Conv2d(64, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (relu): ReLU(inplace=True)
        (downsample): Sequential(
          (0): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
          (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        )
      )
      (1): Bottleneck(
        (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (relu): ReLU(inplace=True)
      )
      (2): Bottleneck(

```

```

    )
    (heads): ModuleDict(
      (expression): Linear(in_features=2048, out_features=8, bias=True)
      (valence): Linear(in_features=2048, out_features=1, bias=True)
      (arousal): Linear(in_features=2048, out_features=1, bias=True)
    )
  )
)

```

In the same way as I modified the classification layer, I added the 3 heads for ResNet as well since our problem requires Multi-head Classification.

Training ResNet-50 with Transfer Learning

Trains the model for up to 50 epochs, but stops early if validation loss stops improving to avoid wasting time.

Two different learning rates are used:

- Backbone (ResNet feature extractor): learning rate ($1e-4$) since we are training on top of a pre-trained model and this part was already trained so a lower learning rate is a safe choice.
- Newly added heads (emotion, valence, arousal outputs): slightly higher learning rate ($1e-3$) so they learn faster since these heads are being trained from scratch.

Three losses are calculated together:

- Cross-entropy for emotion classification (exp)
- MSE loss for valence
- MSE loss for arousal
- All 3 are added to form the total loss.

Uses mixed precision (float16) to speed up training and save GPU memory.

Gradient clipping (clip_grad = 1.0) is applied to stop gradients from exploding and making training unstable.

Tracks training and validation accuracy for classification.

After every epoch, validation metrics are calculated, including:

- Classification: Accuracy, F1, Kappa, Krippendorff's Alpha, ROC-AUC, PR-AUC
- Regression: RMSE, Pearson Correlation, CCC, Sign Agreement (SAGR)

Learning rate is reduced automatically when validation loss stops improving (using ReduceLROnPlateau).

Best validation loss is monitored, and if no further improvement is seen for 10 epochs, training stops. (Early Stopping)

Epoch 11: Train Acc 0.7753, Val Acc 0.4883, F1 0.4859, Kappa 0.4152, Alpha 0.3888, ROC-AUC 0.8398952380952381, PR-AUC 0.49643796592099754

```
Epoch 12/50 [Train]:  0%|          | 0/88 [00:00<?, ?it/s]/tmp/ipykernel_36/1743013057.py:112: Future
Warning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', arg
s...)` instead.
  with torch.cuda.amp.autocast(dtype=torch.float16):
```

Epoch 12: Train Acc 0.8171, Val Acc 0.4833, F1 0.4833, Kappa 0.4095, Alpha 0.2793, ROC-AUC 0.8416571428571429, PR-AUC 0.5005762754455665

```
Epoch 13/50 [Train]:  0%|          | 0/88 [00:00<?, ?it/s]/tmp/ipykernel_36/1743013057.py:112: Future
Warning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', arg
s...)` instead.
  with torch.cuda.amp.autocast(dtype=torch.float16):
```

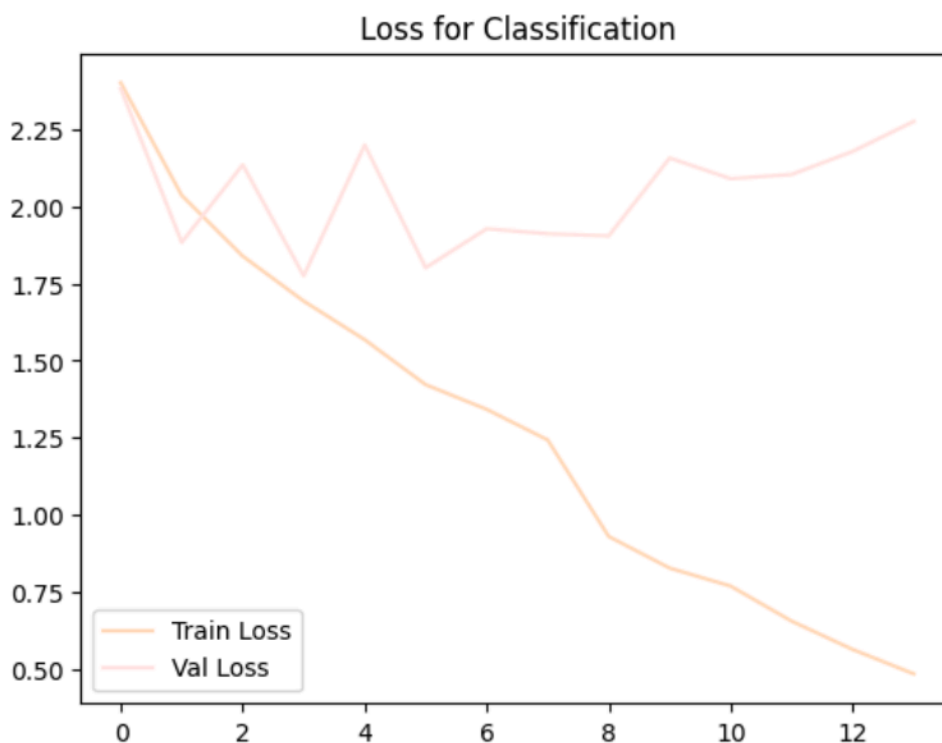
Epoch 13: Train Acc 0.8503, Val Acc 0.5017, F1 0.4965, Kappa 0.4305, Alpha 0.3343, ROC-AUC 0.8471682539682539, PR-AUC 0.5058469728433188

```
Epoch 14/50 [Train]:  0%|          | 0/88 [00:00<?, ?it/s]/tmp/ipykernel_36/1743013057.py:112: Future
Warning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', arg
s...)` instead.
  with torch.cuda.amp.autocast(dtype=torch.float16):
```

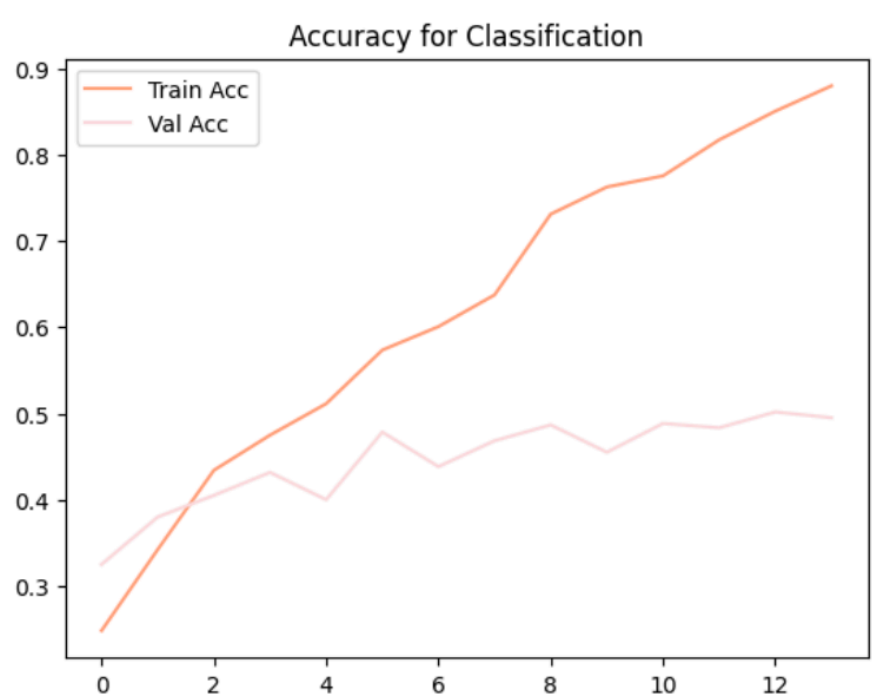
Epoch 14: Train Acc 0.8796, Val Acc 0.4950, F1 0.4936, Kappa 0.4229, Alpha 0.3100, ROC-AUC 0.8483904761904763, PR-AUC 0.5108933793751356

Early stopping at epoch 14

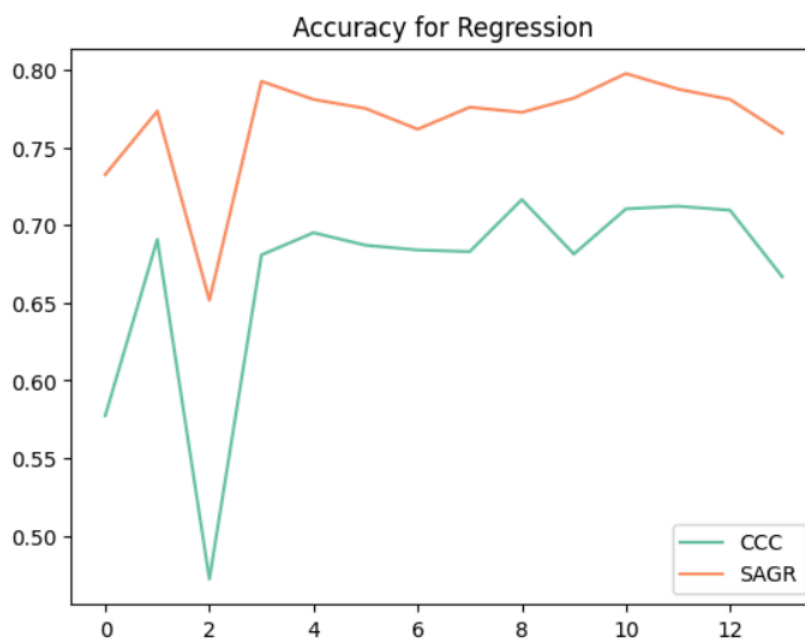
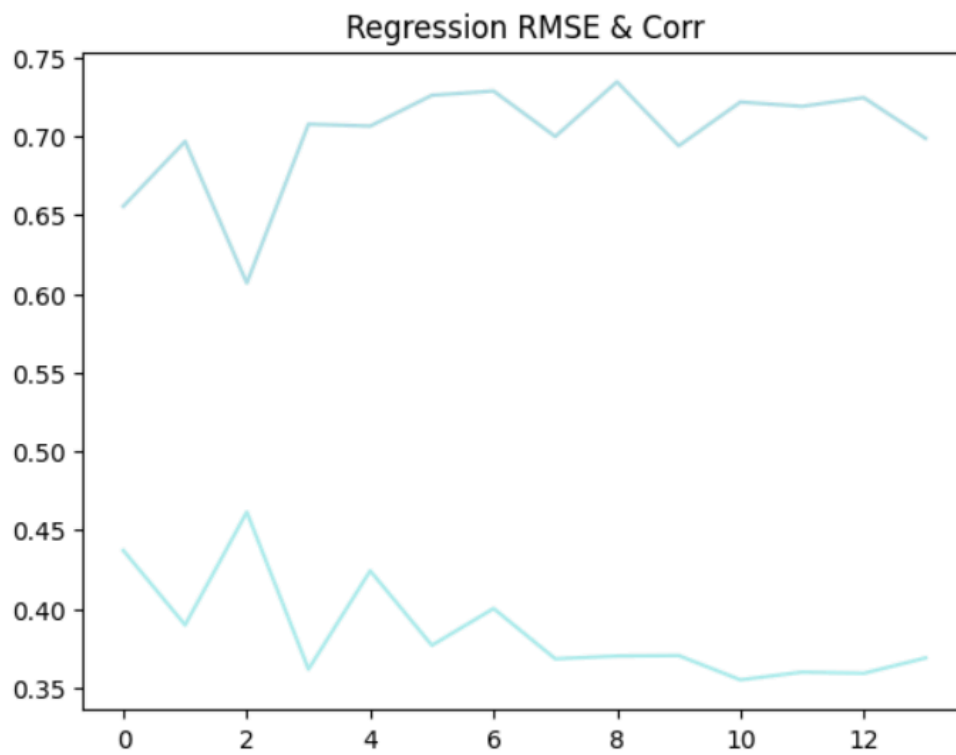
Graphs



Accuracy for Classification



	train_loss	val_loss	train_acc	val_acc	f1	kappa	kripp_alpha	roc_auc	pr_auc
0	2.401048	2.382472	0.248303	0.325000	0.256198	0.228571	-0.050563	0.779130	0.377421
1	2.036045	1.883888	0.342265	0.380000	0.348778	0.291429	0.245566	0.829222	0.444952
2	1.839165	2.135074	0.434084	0.405000	0.376109	0.320000	0.262764	0.827857	0.460229
3	1.693354	1.775389	0.474812	0.431667	0.428608	0.350476	0.266394	0.845987	0.504852
4	1.568209	2.199177	0.511254	0.400000	0.370549	0.314286	0.339583	0.827844	0.453709
5	1.422331	1.801594	0.573419	0.478333	0.454700	0.403810	0.245975	0.855305	0.519460
6	1.342291	1.927729	0.600572	0.438333	0.417804	0.358095	0.223201	0.840130	0.493895
7	1.243122	1.911628	0.637370	0.468333	0.460177	0.392381	0.321664	0.844670	0.504617
8	0.930272	1.904660	0.730975	0.486667	0.486110	0.413333	0.334581	0.853892	0.511641
9	0.827603	2.157192	0.762415	0.455000	0.447398	0.377143	0.326004	0.841768	0.496612
10	0.769282	2.090123	0.775277	0.488333	0.485850	0.415238	0.388787	0.839895	0.496438
11	0.655707	2.103892	0.817078	0.483333	0.483333	0.409524	0.279279	0.841657	0.500576
12	0.563831	2.178603	0.850304	0.501667	0.496473	0.430476	0.334262	0.847168	0.505847
13	0.485022	2.275697	0.879600	0.495000	0.493589	0.422857	0.310040	0.848390	0.510893



	rmse	corr	ccc	sagr
0	0.436986	0.655532	0.577418	0.732500
1	0.389714	0.696727	0.690773	0.773333
2	0.461384	0.606813	0.472229	0.651667
3	0.361626	0.707658	0.680814	0.792500
4	0.424084	0.706428	0.695105	0.780833
5	0.376784	0.725989	0.687026	0.775000
6	0.400168	0.728676	0.683984	0.761667
7	0.368214	0.699813	0.682922	0.775833
8	0.370005	0.734537	0.716429	0.772500
9	0.370295	0.693914	0.681390	0.781667
10	0.354876	0.721705	0.710425	0.797500
11	0.359780	0.718958	0.712162	0.787500
12	0.359035	0.724484	0.709558	0.780833
13	0.368759	0.698694	0.666829	0.759167

Classification Metrics

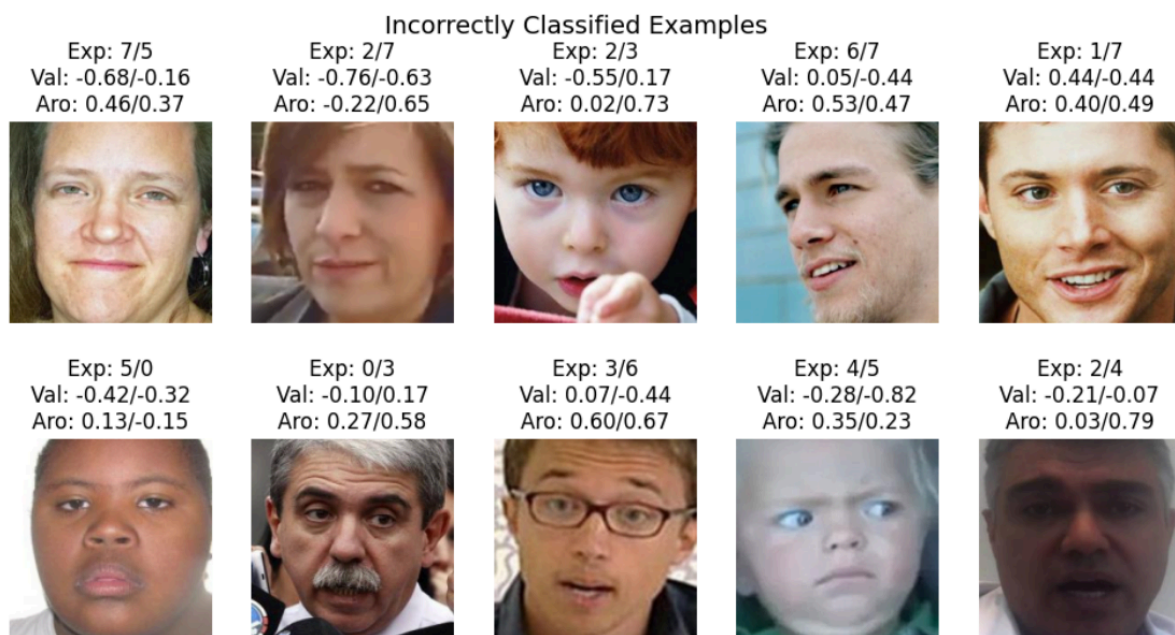
- Training loss consistently decreases, showing the model is learning steadily.
- Training accuracy rises from ~24% to ~88%, indicating the model fits the training data well.
- Validation accuracy peaks around ~50%, which is considered moderate performance on unseen data.
- F1-score follows the same pattern, confirming balanced performance across classes, not just accuracy.

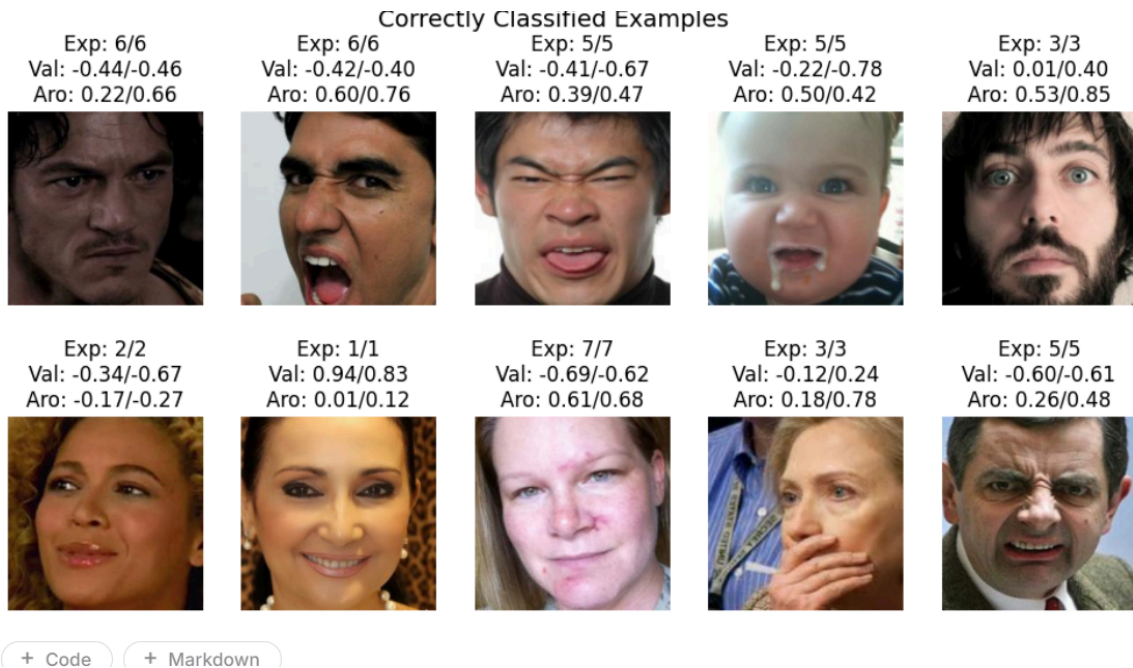
- Cohen's Kappa gradually increases but stays around ~ 0.40 .
- Krippendorff's Alpha remains low-to-moderate ($\sim 0.25\text{--}0.38$).
- ROC-AUC stays high ($\sim 0.83\text{--}0.85$), implying the model is reasonably good at ranking correct vs. incorrect classes.
- PR-AUC improves early and stabilizes around ~ 0.50 , showing moderate precision-recall balance, especially useful for imbalanced classes.

Regression Metrics

- RMSE stays between ~ 0.35 and ~ 0.46 .
- Correlation remains high ($\sim 0.69\text{--}0.73$ across most epochs), meaning the model's predicted trends follow ground truth well.
- CCC (Concordance Correlation Coefficient) reaches ~ 0.70 .
- Sign Agreement (SAGR) stays above ~ 0.75 , meaning in most cases, the model correctly predicts whether emotion intensity is positive or negative.

Model Testing





3. Bonus Part (Base CNN)

Multi-Head CNN Architecture – Why I Built It This Way

- SEBlock (Squeeze-and-Excitation)
 - Inspired by EfficientNet and SENet, I added SEBlocks to let the network learn which channels are important for emotion features.
 - Helps the model focus on subtle facial details like eyes and mouth, which are crucial for expression, valence, and arousal.
- ConvBlock with Residual Shortcut
 - I stacked 2–3 conv layers per block to extract rich features at each stage.
 - Residual connections make the network trainable even at deeper layers and prevent vanishing gradients.
 - Dropout and optional pooling added for regularization and controlling spatial resolution.

- Backbone Design

- I gradually increased channels: $64 \rightarrow 128 \rightarrow 256 \rightarrow 512$ to capture features from low-level edges to high-level emotion patterns.
- Alternating pooled and non-pooled blocks helps preserve spatial information while downsampling gradually.

- Global Average Pooling (GAP)

- Replaces heavy flattening, reduces parameters, and keeps only essential channel-wise information.

- Fully Connected Layer

- Acts as a feature aggregator, combining all backbone information before splitting into task-specific heads.
- Dropout added to reduce overfitting.

- Multi-Head Design

- Each head (expression, valence, arousal) has its own FC layers and dropout.
- Allows the model to learn each task individually while sharing the backbone features efficiently.
- Makes the architecture compact yet powerful for multi-output learning.

- Reasoning

- Inspired by CNN designs like EfficientNet and ResNet.
- SEBlocks + residual connections + multi-head make it attention-aware.

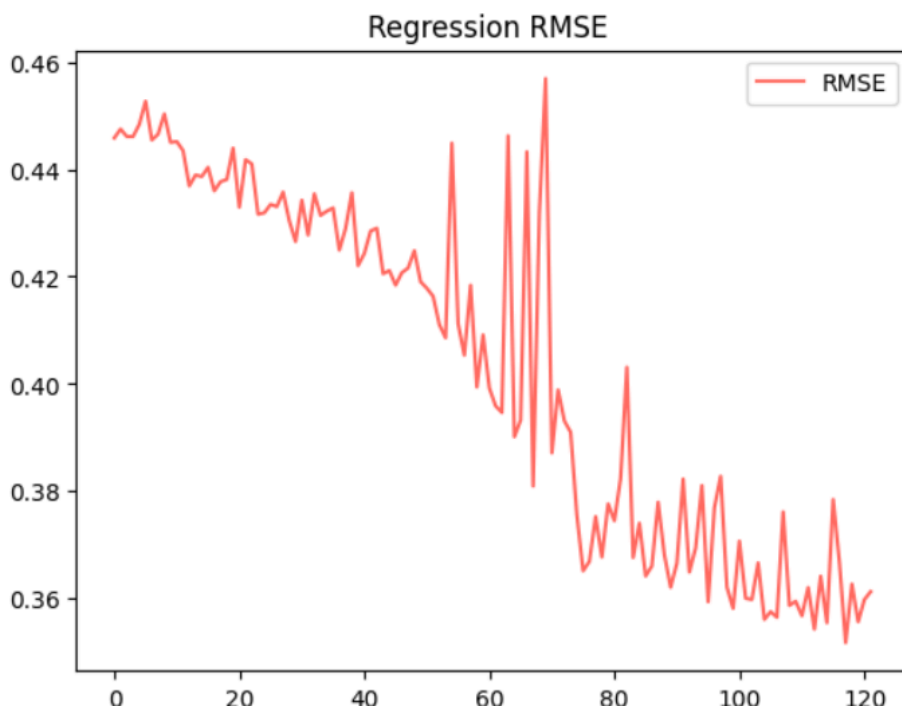
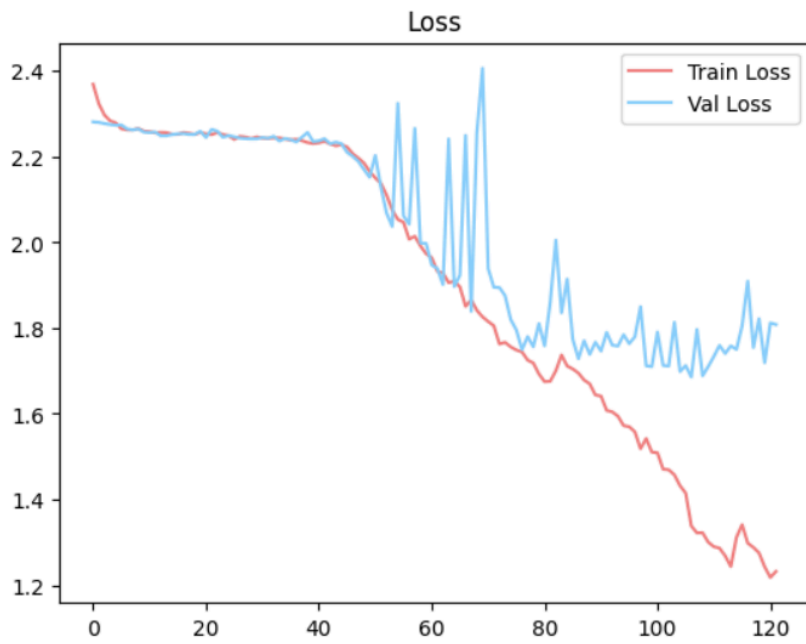
```

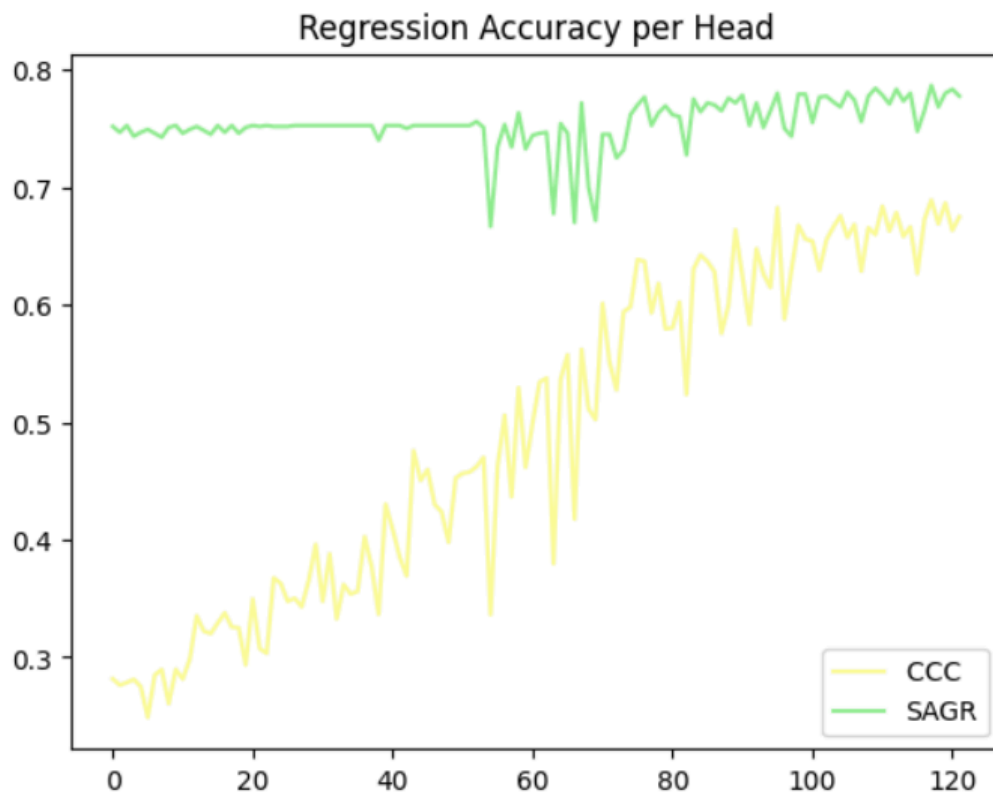
BaseCNNfromscratch(
  (backbone): Sequential(
    (0): ConvBlock(
      (conv): Sequential(
        (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): ReLU(inplace=True)
        (3): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        (4): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (5): ReLU(inplace=True)
      )
      (se): SEBlock(
        (fc1): Linear(in_features=64, out_features=4, bias=False)
        (fc2): Linear(in_features=4, out_features=64, bias=False)
      )
      (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
      (drop): Dropout(p=0.1, inplace=False)
      (shortcut): Conv2d(3, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
    )
    (1): ConvBlock(
      (conv): Sequential(
        (0): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): ReLU(inplace=True)
        (3): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        (4): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (5): ReLU(inplace=True)
      )
      (se): SEBlock(
        (fc1): Linear(in_features=64, out_features=4, bias=False)
        (fc2): Linear(in_features=4, out_features=64, bias=False)
      )
    )

    (fc): Linear(in_features=512, out_features=512, bias=True)
    (drop_fc): Dropout(p=0.5, inplace=False)
    (heads): ModuleDict(
      (expression): Sequential(
        (0): Linear(in_features=512, out_features=256, bias=True)
        (1): ReLU()
        (2): Dropout(p=0.3, inplace=False)
        (3): Linear(in_features=256, out_features=8, bias=True)
      )
      (valence): Sequential(
        (0): Linear(in_features=512, out_features=128, bias=True)
        (1): ReLU()
        (2): Dropout(p=0.2, inplace=False)
        (3): Linear(in_features=128, out_features=1, bias=True)
      )
      (arousal): Sequential(
        (0): Linear(in_features=512, out_features=128, bias=True)
        (1): ReLU()
        (2): Dropout(p=0.2, inplace=False)
        (3): Linear(in_features=128, out_features=1, bias=True)
      )
    )
  )
)

```

Graphs





	train_loss	train_acc	val_loss	val_acc	rmse	corr	ccc	sagr	f1	kappa	kripp_alpha	roc_auc	pr_auc
0	2.369137	0.126116	2.281357	0.128333	0.445859	0.554549	0.281619	0.751667	0.038322	0.003810	-0.147681	0.522875	0.137480
1	2.324031	0.118614	2.280081	0.131667	0.447510	0.552741	0.276337	0.746667	0.047489	0.007619	-0.079885	0.531686	0.150611
2	2.297332	0.126831	2.277341	0.128333	0.446129	0.555248	0.278433	0.752500	0.043445	0.003810	-0.098414	0.531083	0.149679
3	2.283863	0.128617	2.275099	0.140000	0.446125	0.545600	0.281219	0.743333	0.076726	0.017143	-0.041526	0.531556	0.145445
4	2.279008	0.131833	2.273558	0.145000	0.448394	0.546796	0.275025	0.746667	0.096881	0.022857	0.059682	0.547743	0.158249
...	...	...	...	...	...	...	...	...	...	...	...	...	...
17	1.287886	0.571275	1.754578	0.438333	0.351745	0.719451	0.689107	0.786667	0.407560	0.358095	0.278122	0.829781	0.459609
18	1.275054	0.563773	1.821594	0.425000	0.362672	0.709051	0.668800	0.768333	0.401073	0.342857	0.252626	0.819416	0.454951
19	1.242707	0.580922	1.719224	0.453333	0.355638	0.714032	0.686510	0.780000	0.437526	0.375238	0.308283	0.830556	0.473340
20	1.217811	0.584137	1.811868	0.428333	0.359784	0.707704	0.663288	0.783333	0.401133	0.346667	0.220996	0.817298	0.451874
21	1.231751	0.590568	1.808217	0.415000	0.361256	0.705675	0.674794	0.777500	0.385314	0.331429	0.228735	0.817771	0.449325

[100...

	rmse	corr	ccc	sagr
0	0.445859	0.554549	0.281619	0.751667
1	0.447510	0.552741	0.276337	0.746667
2	0.446129	0.555248	0.278433	0.752500
3	0.446125	0.545600	0.281219	0.743333
4	0.448394	0.546796	0.275025	0.746667
...	...	...	...	...
117	0.351745	0.719451	0.689107	0.786667
118	0.362672	0.709051	0.668800	0.768333
119	0.355638	0.714032	0.686510	0.780000
120	0.359784	0.707704	0.663288	0.783333
121	0.361256	0.705675	0.674794	0.777500

122 rows × 4 columns

Training from Scratch

- Optimizer is AdamW with different learning rates for backbone and heads.
- Applied gradient clipping to avoid exploding gradients.
- Used gradient accumulation to simulate larger batch sizes.
- Dropout and weight decay prevent overfitting.
- Losses: cross-entropy for expression, MSE for valence and arousal.

- Weighted losses with lambda values; optional uncertainty weighting for multi-task balancing.
- OneCycleLR scheduler for better learning rate management.
- Early stopping based on validation loss to save the best model.
- Tracked metrics: train/val loss, accuracy, F1, Kappa, ROC-AUC, PR-AUC, RMSE, correlation, CCC, SAGR, Krippendorff alpha.
- Trained for epochs = 160 since the model had no understanding of images etc and was being trained from scratch.
- Used early stopping with patience (default 15) to stop if validation loss doesn't improve.

```
Epoch 98: Train Acc 0.4695, Val Acc 0.4000, F1 0.3731, Kappa 0.3143, ROC-AUC 0.8063015873015873, PR-AUC 0.4164937966487602
```

```
Epoch 99/160 [Train]: 0% | 0/88 [00:00<?, ?it/s]/tmp/ipykernel_36/943437815.py:173: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
with torch.cuda.amp.autocast(enabled=self.device.startswith("cuda")):
```

```
Epoch 99: Train Acc 0.4687, Val Acc 0.4000, F1 0.3696, Kappa 0.3143, ROC-AUC 0.8190190476190475, PR-AUC 0.42673726078454455
```

```
Epoch 100/160 [Train]: 0% | 0/88 [00:00<?, ?it/s]/tmp/ipykernel_36/943437815.py:173: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
with torch.cuda.amp.autocast(enabled=self.device.startswith("cuda")):
```

```
Epoch 100: Train Acc 0.4820, Val Acc 0.3967, F1 0.3633, Kappa 0.3105, ROC-AUC 0.822168253968254, PR-AUC 0.4315234173837704
```

```
Epoch 101/160 [Train]: 0% | 0/88 [00:00<?, ?it/s]/tmp/ipykernel_36/943437815.py:173: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
with torch.cuda.amp.autocast(enabled=self.device.startswith("cuda")):
```

```
Epoch 101: Train Acc 0.4880, Val Acc 0.4117, F1 0.3789, Kappa 0.3276, ROC-AUC 0.8169269841269842, PR-AUC 0.43748955339206463
```

```
Epoch 102/160 [Train]: 0% | 0/88 [00:00<?, ?it/s]/tmp/ipykernel_36/943437815.py:173: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
with torch.cuda.amp.autocast(enabled=self.device.startswith("cuda")):
```

```
Epoch 102: Train Acc 0.4945, Val Acc 0.4067, F1 0.3808, Kappa 0.3219, ROC-AUC 0.8210698412698412, PR-AUC 0.4440125120851902
```

```
Epoch 103/160 [Train]: 0% | 0/88 [00:00<?, ?it/s]/tmp/ipykernel_36/943437815.py:173: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
with torch.cuda.amp.autocast(enabled=self.device.startswith("cuda")):
```

```
Epoch 103: Train Acc 0.4870, Val Acc 0.4100, F1 0.3871, Kappa 0.3257, ROC-AUC 0.8135873015873016, PR-AUC 0.43935567056707215
```

Testing Model

Incorrectly Classified Examples

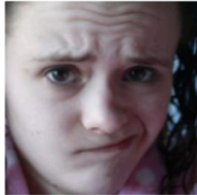
Exp: 0/7
Val: -0.17/-0.22
Aro: 0.29/0.59



Exp: 7/2
Val: -0.29/-0.73
Aro: 0.34/-0.22



Exp: 6/5
Val: -0.32/-0.87
Aro: 0.40/0.46



Exp: 1/3
Val: -0.04/0.26
Aro: 0.32/0.56



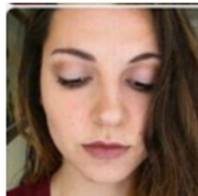
Exp: 7/0
Val: -0.05/0.17
Aro: 0.30/-0.17



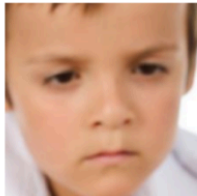
Exp: 4/6
Val: 0.22/-0.48
Aro: 0.62/0.76



Exp: 7/0
Val: -0.13/0.01
Aro: 0.32/0.00



Exp: 2/0
Val: -0.39/-0.06
Aro: -0.04/-0.05



Exp: 1/7
Val: -0.00/-0.65
Aro: 0.24/0.65



Exp: 1/7
Val: 0.52/-0.66
Aro: 0.13/0.66

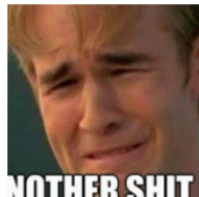


Correctly Classified Examples

Exp: 6/6
Val: -0.39/-0.44
Aro: 0.43/0.71



Exp: 2/2
Val: -0.56/-0.77
Aro: -0.05/-0.30



Exp: 1/1
Val: 0.80/0.46
Aro: 0.09/0.13



Exp: 4/4
Val: -0.26/-0.16
Aro: 0.77/0.87



Exp: 5/5
Val: -0.38/-0.76
Aro: 0.22/0.40



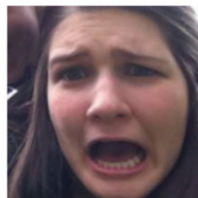
Exp: 1/1
Val: 0.32/0.51
Aro: 0.21/0.09



Exp: 3/3
Val: -0.21/0.29
Aro: 0.42/0.66



Exp: 4/4
Val: -0.27/-0.14
Aro: 0.55/0.81



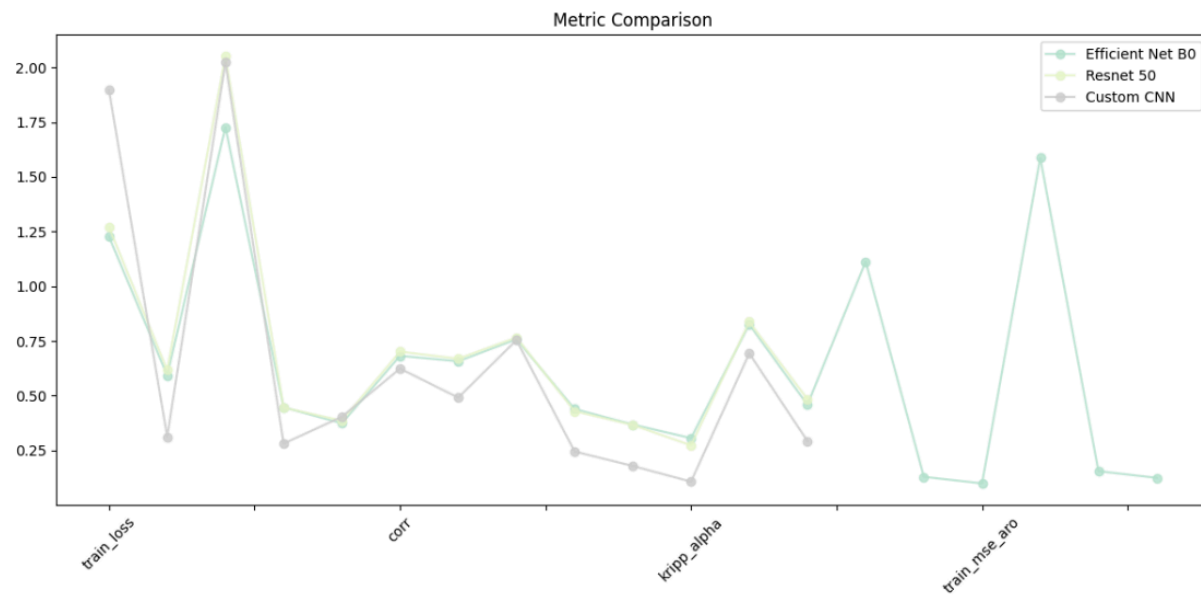
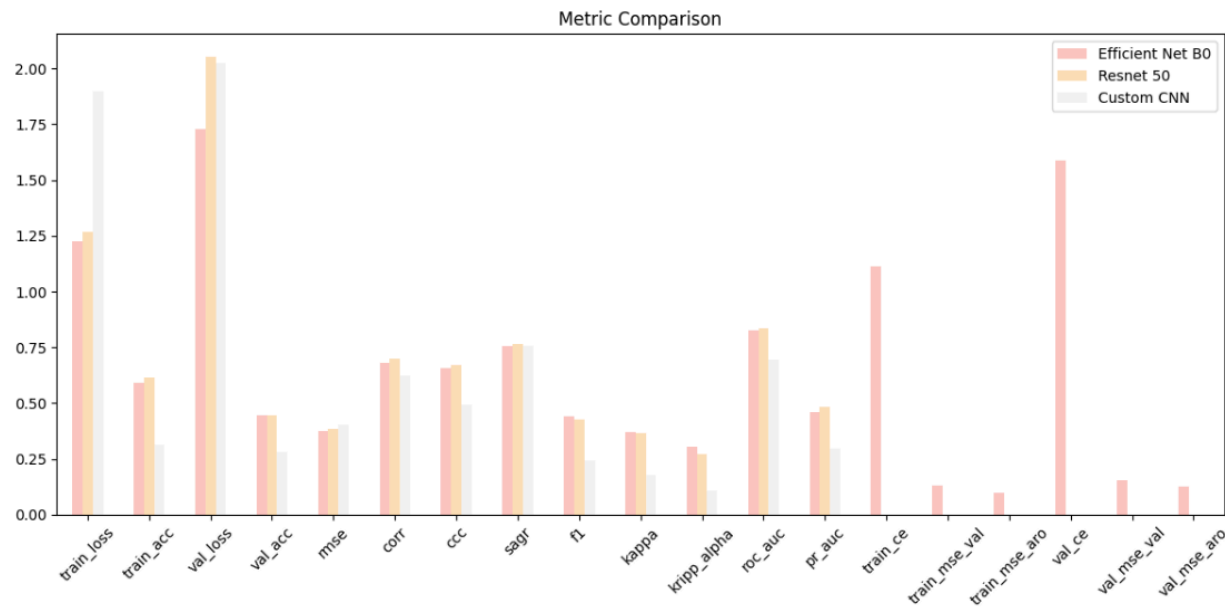
Exp: 6/6
Val: -0.44/-0.22
Aro: 0.48/0.48

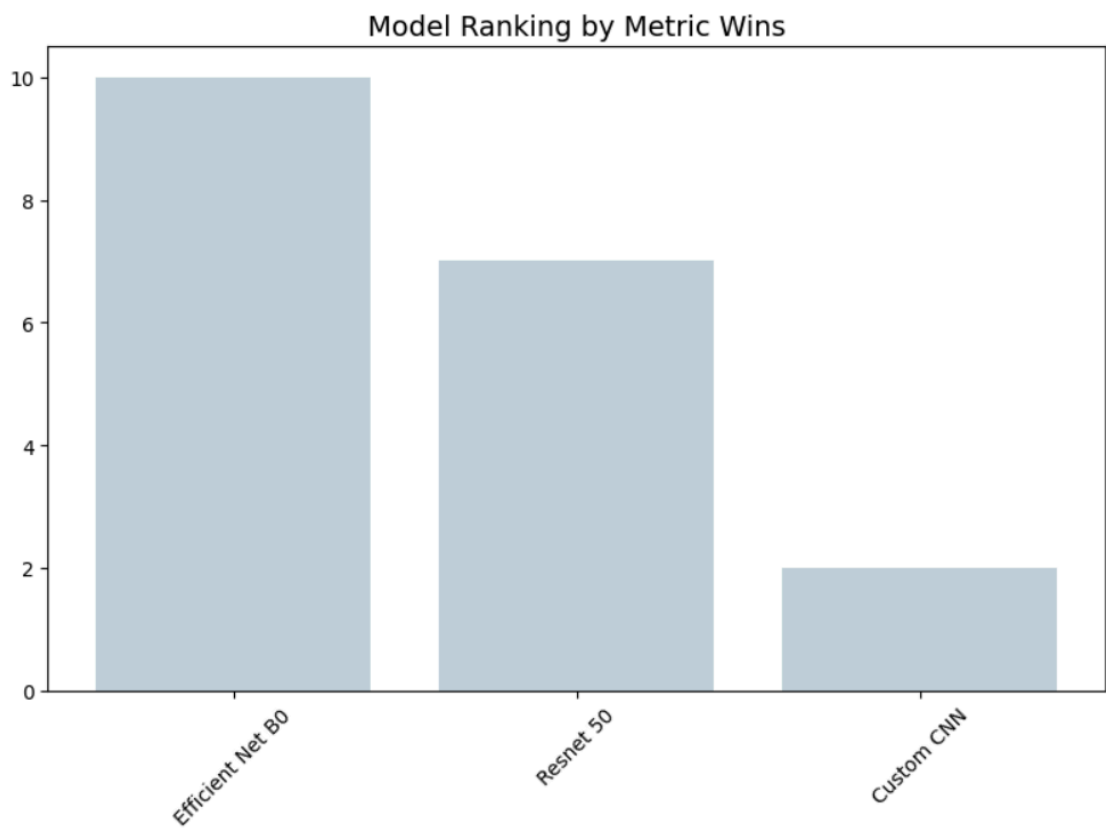
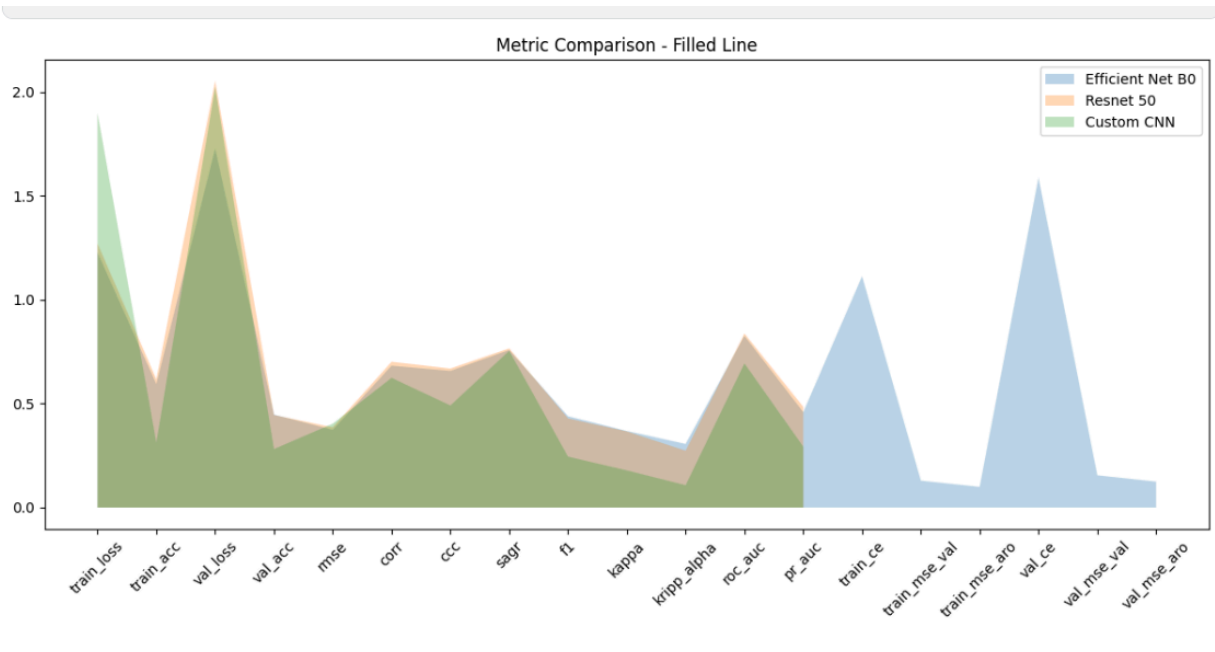


Exp: 5/5
Val: -0.53/-0.59
Aro: 0.42/0.48



Comparing the three models





Links Used for Understanding Different Architectures

<https://www.geeksforgeeks.org/computer-vision/efficientnet-architecture/>

<https://arjun-sarkar786.medium.com/understanding-efficientnet-the-most-powerful-cnn-architecture-eaeb40386fad>

<https://medium.com/@danushidk507/efficientnet-scaling-depth-width-resolution-11e2d4311357>

<https://www.ultralytics.com/blog/what-is-resnet-50-and-what-is-its-relevance-in-computer-vision>

<https://medium.com/@nitishkundu1993/exploring-resnet50-an-in-depth-look-at-the-model-architecture-and-code-implementation-d8d8fa67e46f>

https://medium.com/@sanjay_dutta/designing-your-own-convolutional-neural-network-cnn-model-a-step-by-step-guide-for-beginners-4e8b57836c81

https://www.tensorflow.org/guide/keras/writing_a_training_loop_from_scratch

<https://medium.com/biased-algorithms/creating-a-training-loop-for-pytorch-models-96e260e70766>

<https://debuggercafe.com/multi-head-deep-learning-models-for-multi-label-classification/>